Credit Card Fraud Detection using Machine Learning

Presentation Script

Akshita Modi (AP24110010386) Lakshay (AP24110010677)
Mohana Krishna Reddy Pittu (AP24110011262) Nadimpally Dheva Harshit (AP24110011553)

Supervisor: Mr. Ajay Dilip Kumar Marapatla · SRM University – AP

Total run time: about 9 minutes. Four speakers, split along the work-plan slide. Everything is phrased as work we *will* do. Do not say “we got” or “we found” anywhere.

Slide 1 — Title · 30 sec

Speaker: **Akshita**

Good morning sir. We are presenting our Machine Learning end-semester project proposal – **Credit Card Fraud Detection using Machine Learning**.

Our team is Akshita Modi, Lakshay, Mohana Krishna Reddy Pittu and Nadimpally Dheva Harshit, and we are working under your guidance.

What we want to show you today is the problem we have chosen, why it is harder than it looks, and how we plan to solve it using the algorithms from all five units of the course.

> *Pause. Move to slide 2.*

Slide 2 — Problem Statement · 60 sec

Speaker: **Akshita**

A bank processes millions of card transactions every day, and a very small number of them are fraudulent. Checking each one by hand is impossible, so the flagging has to be automatic.

But the real difficulty here is not the classification itself. It is the **class imbalance**. In real transaction data, fraud is only about zero point one seven percent of all records. That is roughly one fraud in every six hundred transactions.

Because of that, a model can be ninety-nine point eight percent accurate and still catch absolutely nothing.

And the two mistakes are not equal in cost. If we miss a fraud, the bank loses the whole transaction amount. If we raise a false alarm, it only costs one analyst one review. So our goal is to catch as much fraud as possible while keeping false alarms low enough that a real team could actually handle them.

Slide 3 — Abstract · 45 sec

Speaker: **Akshita**

To summarise the whole project in one paragraph –

We will apply the complete Machine Learning syllabus, all five units, to detect fraudulent transactions in a public benchmark dataset of two lakh eighty-four thousand transactions, of which only four hundred ninety-two are fraud.

We will train classifiers taken directly from the course – decision trees, k-nearest neighbour, naive Bayes, SVM, perceptron and neural networks, and ensemble methods – and compare all of them on one common, leakage-free protocol.

Since accuracy is meaningless at this imbalance, we will judge them on recall, precision, F1-score and PR-AUC.

> *Hand over to Mohana.*

Slide 4 — Overview · 45 sec

Speaker: **Mohana**

Just to put the problem in context, sir –

Card fraud losses run into tens of billions of dollars a year worldwide, and they keep growing as digital payments grow. Every bank already runs an automated scoring layer before anything reaches a human, because the analyst team can only review a limited number of alerts per day.

Machine learning is the right tool here for two reasons. Fraud patterns change over time, so fixed if-else rules go stale. And the patterns are hidden in combinations of features, not in a single threshold you can write by hand.

It is also a good project for this course specifically, because it touches almost every algorithm family in our syllabus, and the dataset is public and well studied.

Slide 5 — The challenge, in one number · 45 sec

Speaker: **Mohana**

Sir, this is the slide we would ask you to remember.

Ninety-nine point eight three percent. That is the accuracy of a model that simply predicts “legitimate” for every single transaction. It does not learn anything. It does not look at the data. And it catches zero fraud.

That number is higher than what most published fraud detection papers report as their result.

So this is our justification for not reporting accuracy anywhere in this project. We will report recall, precision, F1-score and the area under the precision-recall curve instead.

> *This is the strongest slide in the deck. Slow down. Let the number sit for two seconds before speaking.*

Slide 6 — Our Unique Solution · 75 sec

Speaker: **Mohana**

So what will make our project different from a standard comparison.

First, one common experimental protocol. Every model sees the same split, the same random seed and the same metrics. Most comparisons you find online change the data between models, so their numbers are not actually comparable.

Second, a strict leakage-free pipeline. We will remove duplicates before splitting, fit the scaler only on training data, and apply any resampling only inside the cross-validation folds – never before the split. This is the mistake most fraud detection projects make, and it is why they report impossibly good numbers.

Third, cost-aware decisions. Instead of using the default zero point five threshold, we will derive the threshold from Bayes optimal decision theory, from Unit III, using the cost of a miss against the cost of a false alarm.

Fourth, breadth. Single learners, ensembles, fixed-rule and trained-rule fusion, and unsupervised clustering – compared head to head.

And everything will be reproducible with a fixed seed of seventy-seven, which is the last two digits of the roll number, as instructed.

Slide 7 — Dataset · 45 sec

Speaker: **Mohana**

The dataset is the ULB credit card fraud dataset, published by the Machine Learning Group at the Université Libre de Bruxelles, and available on Kaggle.

It is real card transaction data from European cardholders over two days in September 2013. Two lakh eighty-four thousand eight hundred and seven transactions, of which four

hundred ninety-two are fraud. That is the zero point one seven percent we mentioned.

There are thirty-one columns. V1 to V28 are PCA-transformed components – the original features were anonymised for privacy reasons. Then Time, Amount, and Class, which is our target.

For preprocessing we plan to remove duplicate records, scale Time and Amount since V1 to V28 are already scaled, and take a stratified seventy-thirty split.

Hand over to Nadimpally.

Slide 8 — Algorithms, Units I to III · 60 sec

Speaker: **Nadimpally**

Sir, these are the algorithms we plan to use, and we have deliberately mapped them to the units of the course.

From **Unit I** we take the methodology itself – proper train, validation and test separation, k-fold cross validation, and an overfitting-underfitting analysis.

From **Unit II**, a decision tree using ID3 with entropy and information gain, and k-nearest neighbour as our instance-based learner. We will also use the feature selection methods – SelectKBest, mutual information and RFE – and dimensionality reduction with PCA and LDA.

From **Unit III**, Gaussian naive Bayes built on Bayes theorem, the Bayes optimal decision threshold derived from a cost matrix, probability calibration, and support vector machines with linear and RBF kernels.

Slide 9 — Algorithms, Units IV and V · 60 sec

Speaker: **Nadimpally**

From **Unit IV**, we start from the McCulloch-Pitts neuron and logic gates, then the perceptron learning rule – including the XOR limitation – and then a multilayer perceptron trained by backpropagation using gradient descent. We also want to apply computational learning theory, VC dimension and PAC bounds, to ask how much data this problem actually needs.

From **Unit V**, bagging, Random Forest, AdaBoost and XGBoost. Then rule fusion – fixed-rule fusion using majority, mean, product, max and min voting, and trained-rule fusion using stacking. And finally K-means and hierarchical clustering, to see whether fraud separates without using the labels at all.

So every unit of the syllabus is represented, sir. That was intentional.

Slide 10 — Tools and Technologies · 40 sec

Speaker: **Nadimpally**

For implementation, Python 3 in Jupyter Notebooks – one notebook per algorithm, so each one stays readable on its own – with Git and GitHub for version control.

NumPy and Pandas for data handling, scikit-learn for the models, pipelines, cross validation and metrics, and the XGBoost library for gradient boosting.

Matplotlib and Seaborn for the confusion matrices, PR and ROC curves, and tree diagrams. And imbalanced-learn for SMOTE and undersampling, applied inside the CV folds.

Our deliverables will be the project report, this presentation, and a public GitHub repository with all the code and results.

Hand over to Lakshay.

Slide 11 — How We Will Do It · 70 sec

Speaker: **Lakshay**

This is our planned workflow, sir, in seven steps.

One, data preparation – load the data, remove duplicates, check integrity.

Two, exploratory analysis – the class distribution, how fraud behaves with respect to amount and time, and the feature correlations.

Three, and this is the important one, we create the split once, stratified, with seed seventy-seven, and every single model reuses that same split. That is what makes the comparison fair.

Four, model building – one notebook per algorithm, each wrapped in a scikit-learn Pipeline so that scaling is fitted inside the fold and cannot leak.

Five, validation with five-fold stratified cross validation, with resampling applied only inside the folds.

Six, threshold tuning from the cost matrix rather than accepting zero point five.

Seven, final comparison – rank every model on the same test set and report what we find.

Slide 12 — How We Will Measure Success · 50 sec

Speaker: **Lakshay**

These are the metrics, and accuracy is deliberately not on the list.

Recall is our primary metric – of all the frauds, how many did we catch. Because a miss is the expensive error.

Precision – of all the alerts we raised, how many were real. This controls how much work we create for the analyst team.

F1-score gives us one balanced number for ranking the models.

PR-AUC, the area under the precision-recall curve, is the best summary metric when the positive class is this rare – better than ROC-AUC, which looks optimistic on imbalanced data.

And the **confusion matrix**, because at the end the bank cares about actual counts of transactions, not percentages.

Slide 13 — Project Goals · 50 sec

Speaker: **Lakshay**

Our goals, sir.

Build a system that catches a high proportion of fraud, while keeping false alarms low enough that a small team could actually review them.

Implement at least one algorithm from every unit, and compare them all on one identical protocol.

Demonstrate clearly why accuracy fails here and which metrics replace it.

Derive the decision threshold from cost using Bayes optimal decision theory.

And then verify the course theory on real data, not just toy examples – we want to show the perceptron actually failing on XOR the way the convergence theorem predicts, and the curse of dimensionality actually degrading k-NN as we add features.

Finally, deliver a fully reproducible repository – fixed seed, pinned versions, and every notebook with its outputs saved.

Slide 14 — Results We Are Expecting · 60 sec

Speaker: **Lakshay**

These are our expectations, sir. We are stating them as hypotheses, because some of them may turn out to be wrong, and we will report that honestly if it happens.

On performance – we expect the ensemble methods, Random Forest and XGBoost, to beat the single learners, with boosting giving the best F1-score. We expect naive Bayes to have good recall but very poor precision, because its independence assumption does

not hold here. And we expect the trivial classifier to hit that ninety-nine point eight three percent while catching nothing.

Our target operating point is recall above zero point eight – catching at least four out of every five frauds – with precision above zero point eight five.

On theory – we expect k-NN to degrade as we add irrelevant features, a single perceptron to fail on XOR while the MLP solves it, and the cost-derived threshold to beat the default zero point five.

And on process, we expect cross validation to confirm the test-set ranking, which would tell us the result is not a fluke.

Slide 15 — Work Plan and Responsibilities · 35 sec

Speaker: **Akshita**

Briefly, how we have divided the work.

Akshita will handle data preparation, the exploratory analysis and the common split. Mohana will take the Unit II and Unit III models – trees, k-NN, naive Bayes, SVM and feature selection. Nadimpally will take Unit IV – the perceptron, the MLP, backpropagation and the learning theory. Lakshay will take Unit V – the ensembles, rule fusion, clustering and threshold tuning.

The final comparison, the report, this presentation and the repository will be done by all four of us together.

Slide 16 — Thank You · 15 sec

Speaker: **Akshita**

That is our proposal, sir. Thank you for listening – we would be glad to take any questions.

Likely questions, and short answers

“Why this dataset and not a bigger or newer one?” It is the standard public benchmark for this problem, it is real transaction data, and it is fully labelled – which is rare for fraud data. Newer datasets are mostly synthetic or not public.

“Why are the features called V1 to V28? What do they mean?” The bank could not release the raw features for privacy reasons, so the publishers applied PCA and released the components. Only Time and Amount are in their original form. We cannot interpret individual components, which we will note as a limitation.

“You have too many algorithms. Why not focus on one?” Because the point of the project is a fair comparison across the syllabus, sir. Running one algorithm well tells us nothing about whether it was the right choice. All of them share the same pipeline, so the additional cost per model is small.

“What is data leakage and where would it happen here?” Leakage is when information from the test set influences training. Here it would happen in three places – if we scale before splitting, if we oversample with SMOTE before splitting, or if duplicate rows end up in both train and test. We handle all three: deduplicate first, split second, and put scaling and resampling inside the Pipeline.

“Why PR-AUC instead of ROC-AUC?” ROC-AUC uses the false positive rate, which has the huge negative class in its denominator. At zero point one seven percent prevalence that denominator swamps everything, so ROC-AUC looks high even for a weak model. PR-AUC uses precision, which reacts to false positives directly.

“How will you pick the threshold?” From Unit III. If a false negative costs us fifty times what a false positive costs, the Bayes optimal threshold is the cost of a false positive divided by the sum of both costs – about zero point zero two, not zero point five. But this only works if the model outputs genuine probabilities, so we will calibrate first and check with a reliability diagram.

“What if a model does not perform as you expect?” Then we report it as it is and explain why, sir. A disproved hypothesis with a clear explanation is a valid result. We would rather report an honest failure than tune the experiment until it agrees with us.

“How is this different from your friend’s / other projects on GitHub?” Most of those train each model on differently prepared data and then report accuracy, which makes the comparison meaningless. Our contribution is the shared protocol, the leakage-free pipeline, the cost-derived threshold, and covering the full syllabus rather than three or four algorithms.